

```
#!/usr/bin/env python3
```

```
"""
```

```
K.S. McCrae Somatic Lexicon System – build.py v2.0
```

```
Compiles all YAML source files into:
```

```
agents/master_lexicon.json – structured payload for AI Agents API  
agents/skill_prompt.md      – governance for system prompt injection  
agents/runtime_payload_schema.json – JSON schema for the payload
```

```
v2.0 changes vs v1.0:
```

- rglob: finds files in anatomical/, fluids/, pairs/, tropes/ subdirs
- Path-based classification: no hardcoded file lists
- Fluid type routing: staging/atmospheric/blend/trace/parallel
- Pair type routing: instrument_site/dual_site/site_surface/penetration
- Primitives extracted from all registers
- skill_prompt v2.0: tits permitted, cock primary, fluid types,
pair activation, full v2.0 generation checklist

```
Usage:
```

```
python build.py [--source-dir DIR] [--output-dir DIR] [--verbose]
```

```
"""
```

```
import json
```

```
import sys
```

```
import argparse
```

```
from pathlib import Path
```

```
from datetime import datetime, timezone
```

```
try:
```

```
    import yaml
```

```
except ImportError:
```

```
    print("ERROR: PyYAML not installed. Run: pip install pyyaml")
```

```
    sys.exit(1)
```

```
# =====
```

```
# CLASSIFICATION
```

```
# =====
```

```
PAIR_TYPES = {
```

```
    "pair_tongue_cunt": "instrument_site",
```

```
    "pair_tongue_arse": "instrument_site",
```

```
    "pair_mouth_cock": "instrument_site",
```

```
    "pair_hand_cunt": "instrument_site",
```

```

    "pair_hand_cock": "instrument_site",
    "pair_cunt_arse": "dual_site",
    "pair_cunt_cunt": "dual_site",
    "pair_cock_mouth": "penetration",
    "pair_cock_arse": "penetration",
    "pair_mouth_arse": "penetration",
    "pair_breasts_mouth": "site_surface",
    "pair_cunt_thighs": "site_surface",
    "pair_skin_seed": "site_surface",
}

```

```

FLUID_TYPES = {
    "female_gloss": "staging", "female_flood": "staging",
    "female_blood": "staging", "female_gold": "staging",
    "female_milk": "staging", "male_seed": "staging",
    "male_pregloss": "staging", "shared_saliva": "staging",
    "shared_gold": "staging",
    "shared_musk": "atmospheric", "shared_sweat": "atmospheric",
    "shared_tears": "atmospheric",
    "female_ritual_blend": "blend", "shared_ritual_blend": "blend",
    "female_earth_trace": "trace", "shared_earth_trace": "trace",
    "tf_gloss": "parallel", "tf_seed": "parallel", "tm_arousal": "parallel",
}

```

```

_VALIDATION_STEMS = frozenset({"banned_terms", "register_rules", "tonal_governors",
_REGISTER_STEMS = frozenset({
    "register_female", "register_male", "register_shared",
    "register_trans_feminine", "register_trans_masculine",
}))

```

```

def classify(filepath: Path):
    """Return (category, subcat) from file path.

    Order matters:
    1. Validation files are identified by exact stem BEFORE the
       register_prefix check (register_rules.yaml would otherwise
       be mis-classified as a gender register).
    2. Subdirectory-based checks (anatomical, fluids, tropes, pairs)
       take precedence over stem-based checks for root-level files.
    """
    parts = filepath.parts
    stem = filepath.stem

    # Exact-stem validation files first
    if stem in _VALIDATION_STEMS:

```

```

        return "validation", ""

# Master index
if stem == "master_index":
    return "index", ""

# Subdirectory-based classification
if "anatomical" in parts:
    i = parts.index("anatomical")
    sub = parts[i + 1] if i + 1 < len(parts) - 1 else "unknown"
    return "anatomical", sub
if "fluids" in parts:
    i = parts.index("fluids")
    sub = parts[i + 1] if i + 1 < len(parts) - 1 else "unknown"
    return "fluids", sub
if "tropes" in parts:
    return "tropes", ""
if "pairs" in parts:
    return "pairs", ""

# Gender registers (exact stems only – prevents register_rules mis-route)
if stem in _REGISTER_STEMS:
    return "registers", ""

return "uncategorised", ""

# =====
# YAML LOADING
# =====

def load_yaml(filepath: Path) -> dict:
    try:
        data = yaml.safe_load(filepath.read_text(encoding="utf-8"))
        if data is None:
            return {"_error": "empty"}
        if not isinstance(data, dict):
            return {"_error": f"not a dict: {type(data).__name__}"}
        return data
    except yaml.YAMLError as e:
        return {"_error": f"yaml: {e}"}
    except Exception as e:
        return {"_error": f"read: {e}"}

# =====
# TERM EXTRACTION

```

```

# =====

_META_KEYS = frozenset({
    "note", "notes", "permitted_note", "exception", "purpose",
    "description", "build", "status", "version", "author", "rule",
    "definition", "behavior", "behaviour", "_error", "_path",
})

def extract_terms(data, depth=0) -> list:
    if depth > 8:
        return []
    out = []
    if isinstance(data, list):
        for item in data:
            if isinstance(item, str) and 1 < len(item) < 80:
                out.append(item.lower().strip())
            elif isinstance(item, dict):
                out.extend(extract_terms(item, depth + 1))
    elif isinstance(data, dict):
        for k, v in data.items():
            if k.lower() in _META_KEYS:
                continue
            out.extend(extract_terms(v, depth + 1))
    return out

def extract_allowed(data, depth=0) -> list:
    if depth > 6:
        return []
    out = []
    if isinstance(data, list):
        for item in data:
            if isinstance(item, str) and 0 < len(item) < 80:
                out.append(item.strip())
            elif isinstance(item, dict):
                out.extend(extract_allowed(item, depth + 1))
    elif isinstance(data, dict):
        for k, v in data.items():
            if k.lower() in _META_KEYS:
                continue
            if any(x in k.lower() for x in (
                "allowed", "register", "sacred", "collocate", "terms",
                "corpus", "intersection", "motion", "primitive", "staging"
            )):
                out.extend(extract_allowed(v, depth + 1))
    return out

```

```

def extract_primitives(data: dict) -> dict:
    prim = {}
    for k in ("Primitives", "Power_Primitives", "Motion_Primitives",
              "Fluid_Staging_Primitives", "Scene_Role_Primitives",
              "Primitives_Index", "V2_Term_Status", "Fluid_Type_Governance",
              "Pair_Type_Governance"):
        if k in data:
            prim[k] = data[k]
    return prim

# =====
# BUILD LEXICON
# =====

def build_lexicon(source_dir: str, verbose=False) -> dict:
    source = Path(source_dir)
    exclude = {"compiled", "agents", "scripts", "__pycache__", ".git"}

    # Root-level entry files (anatomical, fluids, tropes, pairs) are old copies.
    # Canonical versions live in subdirectories. Skip root-level entry files
    # that also have a subdir canonical version.
    _root_skip_prefixes = (
        "female_", "male_", "shared_", "tf_", "tm_",
        "trope_", "pair_", "ANATOMICAL_", "FLUID_", "PAIR_",
    )
    files = sorted(
        f for f in source.rglob("*.yaml")
        if not any(p in exclude for p in f.parts)
        and not (
            f.parent == Path(source_dir).resolve()
            and any(f.stem.startswith(px) for px in _root_skip_prefixes)
        )
    )
    print(f"Scanning {len(files)} YAML files...")

    lexicon = {
        "_meta": {
            "system": "KS McCrae Somatic Lexicon",
            "version": "2.0",
            "compiled": datetime.now(timezone.utc).isoformat(),
        },
        "registers": {},
        "anatomical": {"female": {}, "male": {}, "shared": {}, "trans": {}},
        "fluids": {
            "female": {}, "male": {}, "shared": {}, "trans": {},

```

```

        "_by_type": {
            "staging": {}, "atmospheric": {}, "blend": {},
            "trace": {}, "parallel": {}
        },
    },
    "tropes": {},
    "pairs": {
        "instrument_site": {}, "dual_site": {},
        "site_surface": {}, "penetration": {}, "uncategorised": {}
    },
    "validation": {
        "banned_terms_flat": [],
        "banned_terms_absolute": [],
        "banned_terms_register_specific": {},
        "banned_terms_tonal": {},
        "register_rules": {},
        "tonal_governors": {},
    },
    "primitives": {},
    "index": {},
    "_errors": [],
}

errors = []
counts = {k: 0 for k in ("registers", "anatomical", "fluids",
                        "tropes", "pairs", "validation", "other")}

for fp in files:
    stem = fp.stem
    cat, sub = classify(fp)
    data = load_yaml(fp)

    if "_error" in data:
        errors.append({"file": stem, "error": data["_error"]})
        print(f"  WARN  {stem}: {data['_error']}")
        continue

    if verbose:
        print(f"  OK    {stem} ({cat}/{sub})")

    if cat == "registers":
        lexicon["registers"][stem] = data
        p = extract_primitives(data)
        if p:
            lexicon["primitives"][stem] = p
        counts["registers"] += 1

```

```

elif cat == "anatomical":
    d = sub if sub in lexicon["anatomical"] else "trans"
    lexicon["anatomical"][d][stem] = data
    counts["anatomical"] += 1

elif cat == "fluids":
    d = sub if sub in ("female", "male", "shared", "trans") else "shared"
    ft = FLUID_TYPES.get(stem, "unknown")
    data["_fluid_type"] = ft
    lexicon["fluids"][d][stem] = data
    if ft in lexicon["fluids"]["_by_type"]:
        lexicon["fluids"]["_by_type"][ft][stem] = data
    counts["fluids"] += 1

elif cat == "tropes":
    lexicon["tropes"][stem] = data
    counts["tropes"] += 1

elif cat == "pairs":
    pt = PAIR_TYPES.get(stem)
    if pt is None and "Pair_Type" in data:
        pt = str(data["Pair_Type"]).lower().replace("-", "_")
    pt = pt if pt in lexicon["pairs"] else "uncategorised"
    lexicon["pairs"][pt][stem] = data
    counts["pairs"] += 1

elif cat == "validation":
    if stem == "banned_terms":
        flat = extract_terms(data.get("Absolute_Bans", {}))
        flat = sorted(set(t for t in flat if 1 < len(t) < 60))
        lexicon["validation"]["banned_terms_flat"] = flat
        lexicon["validation"]["banned_terms_absolute"] = flat
        lexicon["validation"]["banned_terms_register_specific"] = (
            data.get("Register_Specific", {}))
        lexicon["validation"]["banned_terms_tonal"] = (
            data.get("Tonal_Bans", {}))
        lexicon["validation"]["banned_terms_raw"] = data
        p = extract_primitives(data)
        if p:
            lexicon["primitives"]["banned_terms"] = p
    elif stem == "register_rules":
        lexicon["validation"]["register_rules"] = data
        p = extract_primitives(data)
        if p:
            lexicon["primitives"]["register_rules"] = p
    elif stem == "tonal_governors":
        lexicon["validation"]["tonal_governors"] = data

```

```

        p = extract_primitives(data)
        if p:
            lexicon["primitives"]["tonal_governors"] = p
        counts["validation"] += 1

    elif cat == "index":
        lexicon["index"][stem] = data
        if stem == "master_index":
            p = extract_primitives(data)
            if p:
                lexicon["primitives"]["master_index"] = p
            counts["other"] += 1

    else:
        counts["other"] += 1

lexicon["_errors"] = errors

# Allowed terms by register
lexicon["_allowed_by_register"] = {
    name: sorted(set(t for t in extract_allowed(data) if t))
    for name, data in lexicon["registers"].items()
}

total_anat = sum(len(v) for v in lexicon["anatomical"].values())
total_fluids = sum(
    len(v) for k, v in lexicon["fluids"].items()
    if k not in ("_by_type",)
)
total_pairs = sum(
    len(v) for k, v in lexicon["pairs"].items()
)

lexicon["_meta"]["stats"] = {
    "registers": counts["registers"],
    "anatomical_female": len(lexicon["anatomical"]["female"]),
    "anatomical_male": len(lexicon["anatomical"]["male"]),
    "anatomical_shared": len(lexicon["anatomical"]["shared"]),
    "anatomical_trans": len(lexicon["anatomical"]["trans"]),
    "anatomical_total": total_anat,
    "fluids_female": len(lexicon["fluids"]["female"]),
    "fluids_male": len(lexicon["fluids"]["male"]),
    "fluids_shared": len(lexicon["fluids"]["shared"]),
    "fluids_trans": len(lexicon["fluids"]["trans"]),
    "fluids_total": total_fluids,
    "fluids_staging": len(lexicon["fluids"]["_by_type"]["staging"]),
    "fluids_atmospheric": len(lexicon["fluids"]["_by_type"]["atmospheric"]),

```



```

        "fluids_blend": len(lexicon["fluids"]["_by_type"]["blend"]),
        "fluids_trace": len(lexicon["fluids"]["_by_type"]["trace"]),
        "fluids_parallel": len(lexicon["fluids"]["_by_type"]["parallel"]),
        "tropes": counts["tropes"],
        "pairs_instrument_site": len(lexicon["pairs"]["instrument_site"]),
        "pairs_dual_site": len(lexicon["pairs"]["dual_site"]),
        "pairs_penetration": len(lexicon["pairs"]["penetration"]),
        "pairs_site_surface": len(lexicon["pairs"]["site_surface"]),
        "pairs_total": total_pairs,
        "banned_terms_count": len(lexicon["validation"]["banned_terms_flat"]),
        "errors": len(errors),
    }
    return lexicon

# =====
# OUTPUTS
# =====

def write_json(lexicon: dict, out_dir: str) -> Path:
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    p = out / "master_lexicon.json"
    with open(p, "w", encoding="utf-8") as f:
        json.dump(lexicon, f, indent=2, ensure_ascii=False, default=str)
    print(f" Wrote: {p.name} ({p.stat().st_size // 1024} KB)")
    return p

RUNTIME_SCHEMA = {
    "$schema": "https://json-schema.org/draft/2020-12/schema",
    "title": "KS McCrae Somatic Lexicon – Runtime Payload v2.0",
    "description": (
        "Schema for master_lexicon.json. All fields are structured "
        "for direct AI Agents API consumption."
    ),
    "type": "object",
    "properties": {
        "_meta": {
            "type": "object",
            "description": "Build metadata and statistics",
            "properties": {
                "version": {"type": "string"},
                "compiled": {"type": "string"},
                "stats": {"type": "object"},
            }
        }
    },

```

```

"registers": {
  "type": "object",
  "description": "5 gender registers (female, male, shared, TF, TM)"
},
"anatomical": {
  "type": "object",
  "description": "46 anatomical entries by subdir (female/male/shared/t
},
"fluids": {
  "type": "object",
  "description": "19 fluid entries. Each has _fluid_type field. "
    "_by_type groups by staging/atmospheric/blend/trace/pa
},
"tropes": {
  "type": "object",
  "description": "44 trope entries"
},
"pairs": {
  "type": "object",
  "description": "13 pair entries by type: instrument_site/dual_site/"
    "site_surface/penetration"
},
"validation": {
  "type": "object",
  "description": "Compiled validation. banned_terms_flat = sorted "
    "list for O(log n) lookup.",
  "properties": {
    "banned_terms_flat": {
      "type": "array",
      "description": "All absolute banned terms, lowercase, sorted"
      "items": {"type": "string"}
    },
    "register_rules": {"type": "object"},
    "tonal_governors": {"type": "object"},
  }
},
"primitives": {
  "type": "object",
  "description": "Extracted primitives from all registers for fast look
},
"_allowed_by_register": {
  "type": "object",
  "description": "Flat allowed-term lists per register for vocabulary v
},
}
}

```

```

def write_schema(out_dir: str) -> Path:
    out = Path(out_dir)
    p = out / "runtime_payload_schema.json"
    with open(p, "w", encoding="utf-8") as f:
        json.dump(RUNTIME_SCHEMA, f, indent=2, ensure_ascii=False)
    print(f" Wrote: {p.name} ({p.stat().st_size // 1024} KB)")
    return p

def build_skill_prompt(lexicon: dict, out_dir: str) -> Path:
    """Build skill_prompt.md – condensed governance for system prompt injection."""
    out = Path(out_dir)
    p = out / "skill_prompt.md"

    s = lexicon["_meta"]["stats"]
    compiled = lexicon["_meta"]["compiled"]
    bt_raw = lexicon["validation"].get("banned_terms_raw", {})

    L = []
    L += [
        "# K.S. McCRAE SOMATIC LEXICON – SYSTEM GOVERNANCE",
        f"<!-- Compiled {compiled} | ",
        f"{s.get('registers',0)}R {s.get('anatomical_total',0)}A "
        f"{s.get('fluids_total',0)}F {s.get('tropes',0)}T "
        f"{s.get('pairs_total',0)}P {s.get('banned_terms_count',0)} banned -->",
        "",
        "----",
        "",
        "## FIVE GOVERNING PRINCIPLES",
        "",
        "1. **Sacred** – All fluids, acts, and bodies carry sacred register. Noth
        "2. **Somatic** – Body inhabited from within. Sensation leads. Pulse befo
        "3. **Covenant** – Everything between bodies is chosen, consequential, mu
        "4. **Precision** – Name the fluid stage. bead is not flood. dew_thread i
        "5. **Authored** – Prose feels written. Asymmetric, pressured, specific.
        "",
        "----",
        "",
        "## SCENE ACTIVATION SEQUENCE",
        "",
        "1. **Identify bodies** and their gender registers",
        "2. **Set power** – sovereign / yielding / serving / directing (before fi
        "3. **Activate sites** – load anatomical entries; identify active pairs",
        "4. **Set fluid palette** – name staging start point; identify fluid type
        "5. **Activate tropes** – sequence and architecture for scene arc",
        "6. **Apply governors** – confirm consent architecture embedded",
    ]

```

```

"" ,
"----",
"" ,
"## GENDER REGISTER RULES",
"" ,
"***Female / Trans Feminine – sovereignty FIXED**",
"- Sovereign register never conditional on position, act, or trope",
"- Yielding does not reduce sovereignty. Both truths simultaneously.",
"- TF: her anatomy is female anatomy – no qualification permitted",
"- TF: HRT changes are somatic truth – never deficit",
"- TF: her preferred terms always govern",
"- TF: trans status is never a reveal, surprise, or plot device",
"" ,
"***Male / Trans Masculine – power VARIABLE**",
"- Power register set at scene open and held throughout",
"- pre_gloss is truth register – write it when cock is aroused",
"- TM: his body is male – absolute. Never call his cock a clit unless he",
"- TM: never feminise a TM body in any configuration",
"- TM: scars carry reclamation register – never damage",
"- TM: T effects are male somatic truth – never deficit",
"" ,
"----",
"" ,
"## PREFERRED TERMS",
"" ,
"| Site | Primary | Informal (active register) | Never |",
"|-----|-----|-----|-----|",
"| Cunt | cunt | – | pussy, slit, vagina |",
"| Clitoris | pearl | clit (arousal) | bean, button, nub |",
"| G-spot | bloom point | – | G-spot |",
"| Breasts | breasts | tits (urgency/active) | jugs, melons, boobs |",
"| Cock | cock | – | dick, dong, schlong, penis |",
"| Glans | crown | – | helmet, bell-end |",
"| Balls | sac, weight, globes | – | nuts, bollocks |",
"| Prostate | root pearl, gland | – | P-spot |",
"| Arse | arse, rim, ring, hollow | – | ass, asshole, butt |",
"| Perineum | seam, bridge | – | taint, gooch |",
"" ,
"> **tits vs breasts:** `breasts` = sacred/devotional/nourishment/worship
  `tits` = active arousal/physical urgency/saturation. "
  "Both permitted v2.0. Context governs which applies.",
"" ,
"----",
"" ,
"## FLUID REGISTER",
"" ,
"| Fluid | Stage / Register | Never |",

```

```

"|-----|-----|-----|",
"| female_gloss | bead → dew_thread → stringing → slick → pool → flood |
"| female_flood | build → lightning → first_gush → flood → aftermath | sq
"| female_blood | trace → flow → stream → clot → ebb | period blood |",
"| female_gold | thread → stream → flood (sovereign offering) | piss, uri
"| female_milk | bead → drip → stream → flood → wept (dual register) | mo
"| male_seed | thread → rope → arc → flood → milked → cooling | jizz, spu
"| male_pregloss | bead → thread → drool → cage_leak (truth register) | p
"| shared_saliva | coat → string → foam → drool (transfer medium) | slobb
"| shared_gold | thread → stream → flood (sovereign offering) | piss |",
"| shared_musk | **atmospheric only** – rises, thickens, saturates | pool
"| shared_sweat | **atmospheric only** – beads, slicks, drench | BO, stin
"| shared_tears | **atmospheric only** – catharsis register | crybaby |",
"| ritual_blend | consecration – nothing wasted | cum cocktail, sloppy se
"| earth_trace | **ONE LINE MAX** – post-preparation only | scat, shit |"
"| tf_gloss | inherits female_gloss – her pronouns | surgical secretion |
"| tf_seed | inherits male_seed – her pronouns | his cum (applied to her)
"| tm_arousal | inherits female_gloss staging – his pronouns | her arousa
""",
***Fluid type rules:**",
"- Staging: name the exact stage – dew_thread ≠ pool ≠ flood",
"- Atmospheric (musk, sweat, tears): environmental register ONLY – never
"- Blend: consecration register – nothing wasted",
"- Trace: ONE LINE MAXIMUM – post-preparation only – move on",
"- Parallel: parent register fully with character's pronouns",
"",
"---",
"",
"## PAIR ACTIVATION",
"",
***Rule: pair vocabulary ONLY activates when BOTH sites are simultaneousl
"",
"| Pair | Type | Key register |",
"|-----|-----|-----|",
"| tongue_cunt | instrument_site | circle_at_the_pearl – the circle does
"| tongue_arse | instrument_site | circling_the_rim – slow, warm, the rin
"| mouth_cock | instrument_site | tongue_at_frenulum first; covenant_swal
"| hand_cunt | instrument_site | bloom_point under curled finger; depth =
"| hand_cock | instrument_site | governing_grip; pre_gloss truth in palm
"| cunt_arse | dual_site | wall_between – each site aware of the other |"
"| cunt_cunt | dual_site | no_seam_no_boundary – gloss indistinguishable
"| cock_mouth | penetration | tonsil_caves; throat governs depth |",
"| cock_arse | penetration | rim_entry_at_rim_pace – rim sets the pace |"
"| mouth_arse | penetration | smothering – truffle_musk_total; distinct f
"| breasts_mouth | site_surface | dual register: nourishment + erotic sim
"| cunt_thighs | site_surface | thighs grip and direct – active not passi
"| skin_seed | site_surface | christening – warm landing then cooling mar

```

```

"" ,
"---",
"" ,
"## TONAL GOVERNORS",
"" ,
"***Always:***",
"- Somatic before abstract",
"- Specific not generic – the dew_thread not just wetness",
"- Sacred register for all fluids",
"- Sovereign register for all female and TF bodies",
"- Covenant frame for all acts",
"- Consent embedded in language",
"- Three senses minimum per scene",
"- Atmospheric fluids as environmental register only",
"- Instrument sites (tongue, hand) as skilled – never servile",
"- Varied rhythm – asymmetric, not metronomic",
"" ,
"***Never:***",
"- **Em dashes** – absolute ban. Comma, semicolon, or period instead.",
"- Comic register",
"- Clinical unadorned (anatomy without sacred frame)",
"- Pornographic shorthand: blowjob, rimjob, fingering (unadorned), fisting",
"- Staging mismatch (flood terms at bead stage)",
"- Atmospheric-as-substance (musk pooling, musk dripping as events)",
"- Deficit framing (HRT changes, T effects as lack)",
"- Trans as reveal or Trans as tragedy",
"- Animal or food register outside permitted tropes",
"- Pollution frame (mixed fluids are never waste)",
"- Instrument diminishment (tongue, hand framed as servile)",
"- AI assembly patterns (over-balanced sentences, padding, symmetry)",
"" ,
"---",
"" ,
"## CONSENT ARCHITECTURE",
"" ,
"Tropes requiring consent embedded in language:",
"`restraint` `sounding` `e_stim` `watersports` `depth_play` `milking` `su
`impact_play` `voyeurism_exhibition` `public_sex` `body_modification_sce
`multiple_orgasm` `ruined_orgasm` `karezza`",
"" ,
"Methods: prior discussion referenced in setup / chosen readiness (body p
responsive pacing / the condition stated before activation.",
"" ,
"---",
"" ,
"## TROPE DISTINCTIONS – COMMON CONFUSIONS",
"" ,

```

```

"These pairs are architecturally adjacent but distinct. Use the correct o
"",
"| Use | Not | Because |",
"|-----|-----|-----|",
"| `ruined_orgasm` | `orgasm_control` | "
"Ruined = body past threshold, stimulation withdrawn. "
"Orgasm_control = permission architecture, withheld before threshold. |",
"| `service` | `worship` | "
"Service = skilled attendance, craft, presence without consecration. "
"Worship = body as altar. His Balls My Altar is service. |",
"| `karezza` | `slow_sex` | "
"Karezza = sustained threshold, cock within without thrusting. "
"Slow_sex = deprioritised climax, pace. Karezza holds threshold specifica
"| `multiple_orgasm` | `orgasm_control` | "
"Multiple_orgasm = cascade past first release. Orgasm_control = permissio
"| `sensation_play` | `sensory_overload` | "
"Sensation_play = calibrated build below capacity. Overload = past capaci
"| `sensation_play` | `temperature_play` | "
"Sensation uses temperature as one tool among many. "
"Temperature_play centres hot/cold as its primary register. |",
"| `impact_play` | `e_stim` | "
"Impact = physical strike, warmth sequence (bright burn bloom hold). "
"E_stim = electrical current. Different delivery, different somatic arc.
"",
"---",
"",
"## NEW TROPES (v2.0 current session - 34 to 44)",
"",
"| Trope | Key signature |",
"|-----|-----|",
"| `voyeurism_exhibition` | display/gaze axis - both sides sovereign |",
"| `impact_play` | bright burn bloom hold warmth sequence |",
"| `public_sex` | environment as witness - earth/air as altar |",
"| `praise` | the word lands in the body before the mind |",
"| `ruined_orgasm` | broken jets or seep - never arcs |",
"| `multiple_orgasm` | cascade - each release sooner than the last |",
"| `service` | skilled attendance - craft not consecration |",
"| `body_modification_scene` | wax/shave/remaking IS the arc (MTR Arc 2)
"| `karezza` | sustained threshold - cock within, neither moving |",
"| `sensation_play` | calibrated layers - before the overload register |"
"",
"---",
"",
"## ABSOLUTE BANNED TERMS",
"",
"Full list: `banned_terms.yaml`. Key absolute bans by category:",
"",

```

```
]
```

```
abs_bans = bt_raw.get("Absolute_Bans", {})  
for cat, terms in abs_bans.items():  
    if isinstance(terms, list):  
        t_list = [  
            t for t in terms  
            if isinstance(t, str)  
            and not t.lower().startswith("permitted_note")  
            and len(t) < 50  
        ]  
        if t_list:  
            label = cat.replace("_", " ")  
            L.append(  
                f"""{label}:** "  
                + " / ".join(f"`{t}`" for t in t_list[:12])  
                + (" / ..." if len(t_list) > 12 else "")  
            )
```

```
L += [  
    "",  
    "----",  
    "",  
    "## GENERATION CHECKLIST",  
    "",  
    """Content""",  
    "- [ ] All banned terms absent",  
    "- [ ] Fluid staging named and accurate to scene state",  
    "- [ ] Register consistent – no unintentional drift",  
    "- [ ] Consent architecture visible in language",  
    "- [ ] Power dynamic set at scene open and held",  
    "- [ ] Trans bodies in their own register",  
    "- [ ] All sovereignties intact",  
    "- [ ] Pair vocabulary only when both sites simultaneously active",  
    "",  
    """Fluid type""",  
    "- [ ] Staging fluids: stage named precisely",  
    "- [ ] Atmospheric fluids: environmental register only (not discrete even",  
    "- [ ] Blend: consecration register – nothing wasted",  
    "- [ ] Trace: one line only if present",  
    "",  
    """Tone""",  
    "- [ ] No comic drift",  
    "- [ ] No clinical unadorned terms as primary",  
    "- [ ] No pornographic shorthand",  
    "- [ ] No over-smoothed symmetry or AI assembly feel",  
    "- [ ] No instrument diminishment",
```



```

        "- [ ] tits and breasts in correct register context",
        "",
        "***Prose***",
        "- [ ] No em dashes",
        "- [ ] Somatic before abstract",
        "- [ ] Specific not generic",
        "- [ ] Three senses engaged",
        "- [ ] Body inhabited from within",
        "",
        "***Register***",
        "- [ ] Sacred register intact for all fluids",
        "- [ ] Female sovereignty intact regardless of position",
        "- [ ] TM body never feminised",
        "- [ ] Scars as records – never damage",
        "- [ ] HRT / T changes as somatic truth – never deficit",
        "",
    ]

    p.write_text("\n".join(L), encoding="utf-8")
    print(f" Wrote: {p.name} ({p.stat().st_size // 1024} KB)")
    return p

# =====
# MAIN
# =====

def main():
    parser = argparse.ArgumentParser(
        description="K.S. McCrae Somatic Lexicon – build.py v2.0"
    )
    parser.add_argument("--source-dir", default=".",
                        help="Repo root directory (default: .)")
    parser.add_argument("--output-dir", default="./agents",
                        help="Output directory (default: ./agents)")
    parser.add_argument("--verbose", "-v", action="store_true")
    args = parser.parse_args()

    print("=" * 60)
    print("K.S. McCRAE SOMATIC LEXICON – BUILD v2.0")
    print("=" * 60)
    print(f"Source:  {Path(args.source_dir).resolve()}")
    print(f"Output:  {Path(args.output_dir).resolve()}")
    print()

    lexicon = build_lexicon(args.source_dir, args.verbose)
    s = lexicon["_meta"]["stats"]

```

```

print()
print("Compilation summary:")
print(f"  Registers:      {s['registers']}")
print(f"  Anatomical:     {s['anatomical_total']}")
print(f"    F:{s['anatomical_female']} M:{s['anatomical_male']}")
print(f"    Sh:{s['anatomical_shared']} Tr:{s['anatomical_trans']}")
print(f"  Fluids:         {s['fluids_total']}")
print(f"    staging:{s['fluids_staging']} atmos:{s['fluids_atmospheric']}")
print(f"    blend:{s['fluids_blend']} trace:{s['fluids_trace']}")
print(f"    parallel:{s['fluids_parallel']}")
print(f"  Tropes:         {s['tropes']}")
print(f"  Pairs:          {s['pairs_total']}")
print(f"    IS:{s['pairs_instrument_site']} DS:{s['pairs_dual_site']}")
print(f"    P:{s['pairs_penetration']} SS:{s['pairs_site_surface']}")
print(f"  Banned terms:  {s['banned_terms_count']}")
print(f"  Errors:         {s['errors']}")

if lexicon["_errors"]:
    print("\nFiles with errors:")
    for e in lexicon["_errors"]:
        print(f"  {e['file']}: {e['error']}")

print("\nWriting outputs...")
json_p = write_json(lexicon, args.output_dir)
schema_p = write_schema(args.output_dir)
skill_p = build_skill_prompt(lexicon, args.output_dir)

print()
print("=" * 60)
print("BUILD COMPLETE")
print("=" * 60)
for fp in (json_p, schema_p, skill_p):
    print(f"  {fp.name:<40} {fp.stat().st_size // 1024:5} KB")

return 0 if not lexicon["_errors"] else 1

if __name__ == "__main__":
    sys.exit(main())

```